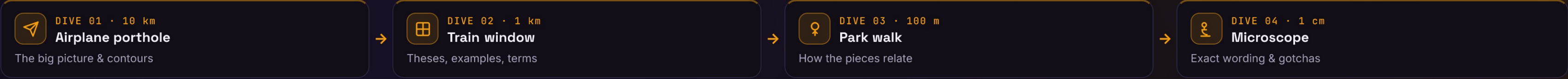




Adapter

Convert one class's interface into the interface a client expects — so incompatible code can work together.



Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

Adapter wraps an existing class in the interface a client expects — translating calls, so incompatible code works together unchanged.

THE CONTOURS

• Translate, don't rewrite

wrap a class so its interface matches what callers expect

• Reuse the incompatible

fit legacy or third-party code into your own contract

• The travel-plug pattern

— convert one socket shape into another, nothing more

USE IT WHEN — GoF applicability

🎯 Existing class, wrong interface

you want to reuse it, but its interface doesn't match what you need

🔄 Cooperate with the unforeseen

a reusable class must work with classes it wasn't designed for

📦 Structural family — Adapter changes an interface to match expectations; Decorator keeps the interface and adds behavior; Facade gives a new, simpler interface to a whole subsystem.

↓ DIVE DEEPER

Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

Target

the interface the client codes against (request())

Adaptee

the existing class with a useful but mismatched API

Adapter

implements Target, delegates to the Adaptee's methods

IN THE WILD

✳️ Arrays.asList(arr)

adapts a raw array to the List interface

📄 InputStreamReader

adapts a byte stream to a char Reader (JDK)

⌌ HTTP client wrappers

wrap axios / fetch behind your own API

💳 Payment SDK adapters

one PaymentGateway over Stripe / PayPal SDKs

⌌ Client → Target.request() → Adapter → Adaptee.specificRequest() — the translated call.

GoF intent: "Convert the interface of a class into another interface clients expect."

↓ DIVE DEEPER

Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Client

wants Target

→

Adapter.request()

the wrapper

→

translate the call

map names & params

→

✓ Adaptee.specificRequest()

real work done

WHY IT FOLLOWS

→ Adapter implements Target ⇒ the client sees exactly the interface it already expects

→ Adapter holds an Adaptee ⇒ it delegates, converting names, parameters and types

→ Wrap, don't modify ⇒ the Adaptee — legacy or third-party — stays untouched

🔗 Decorator and Proxy keep the same interface (Adapter changes it); Facade adapts a whole subsystem; Bridge looks similar but is designed up front, not retrofitted.

STRUCTURE · UML

Adapter

- adaptee: Adaptee

+ request(): void

Adapter implements Target.request() by delegating to adaptee.specificRequest(), converting the call. The client depends only on Target.

WHAT IT BUYS YOU

✔ Reuses existing code — wrap a class you can't or shouldn't modify

✔ Decouples client & Adaptee — the client depends only on the Target interface

✔ Object adapter is flexible — one adapter can wrap an Adaptee and its subclasses

✔ Single responsibility — conversion logic lives in one isolated place

✔ Two flavors — object adapter (composition) vs class adapter (inheritance)

↓ DIVE DEEPER

Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: "Convert the interface of a class into another interface clients expect; Adapter lets classes work together that otherwise couldn't because of incompatible interfaces."

— GoF, "Design Patterns", 1994

THE CODE

```
interface Target { request(): string; }
class LegacyService { // Adaptee
  specificRequest() { return "raw-data"; }
}
class Adapter implements Target {
  constructor(private a: LegacyService) {}
  request() {
    return normalize(this.a.specificRequest());
  }
}
```

VARIANTS

Object adapter

composition — holds an Adaptee; the GoF default

Class adapter

subclass the Adaptee + implement Target (not in JS)

Two-way adapter

implements both interfaces; usable as either

Pluggable adapter

parameterize the adapter with the mapping

Default adapter

empty impls of a wide interface; override a few

Function adapter

wrap / curry a function to a new signature (JS)

COMPARE THE ALTERNATIVES

Aspect	Adapter	Decorator	Facade	Proxy
Changes the interface	✓	✗	✓	✗
Adds new behavior	~	✓	✗	~
Wraps one object	✓	✓	✗	✓
Simplifies a subsystem	✗	✗	✓	✗
Retrofitted after the fact	✓	~	✓	~

Best fit → Adapter: fit a mismatched interface · Decorator: add behavior · Facade: simplify a subsystem · Proxy: control access. ("~" = sometimes.)

INTERVIEW PITFALLS

Q "Adapter vs Decorator?"

Both wrap an object, but Adapter changes the interface to a different one; Decorator keeps the same interface and layers on behavior.

Q "Object vs class adapter?"

Object adapter composes the Adaptee (adapts its subclasses too); class adapter subclasses the Adaptee and implements Target — fine in Java, but impossible in JS. Prefer the object form.

Q "Adapter vs Facade?"

Adapter makes one existing class fit an expected interface; Facade invents a new, simpler interface over a whole subsystem.

Q "Where do you meet it daily?"

Wrapping third-party SDKs and legacy APIs behind your own port, so the app depends on your interface, not theirs.

NUANCES & GOTCHAS

• Object vs class adapter — composition adapts an Adaptee and its subclasses; subclassing binds to one Adaptee

• Adapter vs Decorator — Adapter changes the interface; Decorator keeps it and adds behavior

• Adapter vs Facade — Adapter retrofits one interface; Facade invents a simpler one over many classes

• Adapter vs Bridge — Bridge is planned to vary two sides; Adapter is added after the fact to fix a mismatch

• Keep adapters thin — translate only; business logic belongs in the Adaptee or the domain

• Two-way needs care — supporting both directions can leak each interface's assumptions

WATCH OUT

⚠ Anti-pattern: Don't adapt when you can change the source — fixing the interface directly is cleaner. Avoid stacking adapters over adapters; that signals a missing abstraction, not a translation problem.

⚡ Modern take: in JS a thin wrapper module or a mapping function is the everyday adapter — used constantly to fit third-party SDKs and legacy APIs behind your own ports (hexagonal architecture).

• VasyI Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF "Design Patterns" (1994) v1